

Travailler avec l'application

Installer. L'application demande aussi Node.js 20 ou plus, avec npm (par exemple `mise en place --global node@lts`), et SQLite 3 (`sqlite3 --version`). Elle se clone **dans** le dossier des supports : les commandes `cp` des steps en dépendent.

```
cd formation-ruby
git clone https://github.com/Tuxii/workshops.git
cd workshops
bin/setup --skip-server          # installe les gems et les paquets npm, crée la base
bin/rails runner 'puts Rails.version' # affiche 8.1.4
bin/dev                          # puis http://localhost:3000 ; Ctrl+C pour arrêter
```

Les branches. `workshops/` a une branche par step : `step-NN` contient l'application **à la fin** de l'étape NN, c'est la solution ; `step-00` est le point de départ. Vous travaillez sur votre propre branche : `git switch -c mon-travail origin/step-00`.

Les consignes sont dans `rails/steps/`. La section « Step NN » de ce support accompagne le step NN : ouvrez-la quand la consigne demande une syntaxe, relisez-la en fin de step.

Les fichiers fournis. Depuis `workshops/`, lancez la commande `cp` donnée par l'énoncé, au moment où il la donne, par exemple `cp -r ../rails/fournis/step-NN/. .`. La copie écrase les fichiers de même nom : committez avant, `git diff` montrera ensuite ce qui a changé.

Comparer avec la solution. Les solutions sont publiées au fil de la formation :

```
git fetch                    # récupère les solutions publiées
git diff origin/step-NN     # compare votre code avec la solution
git diff origin/step-NN -- app/models # seulement les modèles
git show origin/step-NN:app/models/session.rb # affiche un fichier de la solution
```

Repartir d'une solution. Mettez d'abord votre travail de côté, sinon `git switch` refusera de changer de branche :

```
git add -A && git commit -m "Travail en cours" # ou : git stash -u
git switch -c rattrapage-NN origin/step-NN
bundle install                                # si la branche utilise d'autres gems
bin/rails db:reset                            # recrée la base, rejoue les seeds
bin/dev                                       # après Ctrl+C sur le bin/dev en cours
```

Lancez `bin/rails db:reset` à chaque changement de branche, puis relancez `bin/dev` : sinon le serveur garde ouverte l'ancienne base supprimée et affiche des données périmées. N'utilisez pas `bin/setup` ni `bin/rails db:prepare` à la place : ils lanceraient les migrations de la branche sur votre base existante et s'arrêteraient sur `table "workshops" already exists`.

Pour retrouver votre travail : `git switch mon-travail`, `git stash pop` si vous avez utilisé `git stash -u`, puis `bin/rails db:reset` et `bin/dev` relancé.

Les exemples de ce support décrivent une bibliothèque : des auteurs (`Author`) et leurs livres (`Book`), chacun en un nombre limité d'exemplaires (`copies`), des membres (`Member`), leurs emprunts (`Loan`) et des avis (`Review`).

♦ Step 01 : l'application et Active Record

Structure et conventions de nommage

app/	le code, rangé par rôle : models/, controllers/, views/, helpers/...
config/routes.rb	toutes les routes, dans un seul fichier
config/	la configuration : environments/, initializers/, locales/...
db/	migrate/ (les migrations), schema.rb (l'état du schéma), seeds.rb
test/	les tests
Gemfile	les dépendances, gérées par Bundler ; versions figées dans Gemfile.lock
bin/rails	la ligne de commande : générateurs, migrations, console, serveur

Un nom suffit à retrouver tout le reste :

Book	le modèle, dans app/models/book.rb
books	sa table : au pluriel, en snake_case
book_id	une clé étrangère vers un livre
BooksController	son contrôleur, dans app/controllers/books_controller.rb
app/views/books/	les vues de ce contrôleur : index.html.erb, show.html.erb...
resources :books	ses routes, et leurs helpers : books_path, book_path(book)...

Un nom composé suit la même règle (`LoanRequest`, table `loan_requests`, fichier `loan_request.rb`), avec le pluriel de l'anglais. En cas de doute : `"Person".tableize` renvoie `"people"`.

Migrations et `db/schema.rb`

```
bin/rails generate model Book title:string
```

```
class CreateBooks < ActiveRecord::Migration[8.1]
  def change
    create_table :books do |t|
      t.string :title
      t.timestamps # created_at et updated_at, remplies par Rails
    end
  end
end
```

La clé primaire `id` est implicite. `bin/rails db:migrate` joue les migrations en attente, puis réécrit `db/schema.rb` en lisant la base : ce fichier décrit l'état courant du schéma et ne se modifie jamais à la main.

Le modèle ne déclare rien (`class Book < ApplicationRecord; end`). Active Record lit les colonnes de la table à l'exécution, et génère les méthodes d'accès (`title`, `title=` ...) dans un module inclus dans la classe. Les attributs d'un modèle se lisent donc dans `db/schema.rb`, pas dans le fichier du modèle.

Créer, retrouver, modifier, supprimer

```
book = Book.new(title: "Germinal") # aucun SQL : un objet en mémoire, id vaut nil
book.save                         # INSERT INTO "books" (...) VALUES (...) RETURNING "id"
Book.create(title: "Nana")        # new, puis save
Book.find(3)                     # SELECT "books".* FROM "books" WHERE "books"."id" = 3 LIMIT 1
                                # lève ActiveRecord::RecordNotFound si rien n'est trouvé
Book.find_by(title: "Nana")       # ... WHERE "books"."title" = 'Nana' LIMIT 1, ou nil
Book.where(title: "Nana")         # une relation : SQL seulement quand on la parcourt
Book.order(:title)               # ... ORDER BY "books"."title" ASC
book.update(title: "Nana (poche)") # UPDATE "books" SET "title" = ..., "updated_at" = ...
book.destroy                     # DELETE FROM "books" WHERE "books"."id" = 3
                                # l'objet Ruby reste, gelé
book.reload                      # relit la ligne en base ; RecordNotFound si elle a disparu
```

En console, `Book.where(...)` affiche pourtant un `SELECT` : c'est l'affichage du résultat qui parcourt la relation. `relation = Book.where(title: "Nana"); nil` n'exécute rien.

save écrit tout de suite. Il envoie la requête aussitôt, dans sa propre transaction : le log montre `BEGIN`, l'`INSERT` ou l'`UPDATE`, puis `COMMIT`. L'objet connaît ses modifications, et `save` n'écrit que les colonnes modifiées : après `book.title = "Germinal (poche)"`, `book.changed?` renvoie `true` et `book.changes` renvoie `{"title" => ["Germinal", "Germinal (poche)"]}`.

La console et le SQL dans les logs. `bin/rails console` charge l'application ; `bin/rails console --sandbox` annule tout à la sortie. En développement, la console affiche chaque requête SQL sous l'appel qui l'a déclenchée, `reload!` recharge le code modifié, et `relation.to_sql` montre le SQL d'une relation sans l'exécuter. Le serveur et les tests écrivent les mêmes lignes dans `log/development.log` et `log/test.log`.

♦ Step 02 : migrations, validations, chargement des classes

Faire évoluer une table

Chaque changement de schéma est une nouvelle migration. Le nom `Add...ToBooks` indique la table au générateur :

```
bin/rails generate migration AddDetailsToBooks pages:integer copies:integer
```

```
class AddDetailsToBooks < ActiveRecord::Migration[8.1]
  def change
    add_column :books, :pages, :integer
    add_column :books, :copies, :integer, default: 1, null: false # options ajoutées à la main
  end
end
```

Rails sait inverser `add_column`, `create_table` ou `add_index` : avec une méthode `change`, la migration est réversible. Une migration déjà jouée ailleurs ne se modifie plus.

```
bin/rails db:migrate          # joue les migrations en attente, réécrit db/schema.rb
bin/rails db:migrate:status   # up ou down, pour chaque migration
bin/rails db:rollback         # défait la dernière
bin/rails db:reset            # recrée la base depuis db/schema.rb, puis joue les seeds
```

Validations et errors

```
class Book < ApplicationRecord
  validates :title, presence: true, length: { maximum: 200 }
  validates :pages, numericality: { only_integer: true, greater_than: 0 }, allow_nil: true
end

book = Book.new(title: "")
book.valid?          # => false : exécute les validations et remplit errors
book.errors.full_messages # => ["Title doit être rempli(e)"]
book.errors[:title]   # => ["doit être rempli(e)"]
book.save             # => false, rien n'est écrit
book.save!            # lève ActiveRecord::RecordInvalid
```

Même convention partout : sans `!`, la méthode renvoie `false` ; avec `!` (`save!`, `create!`, `update!`), elle lève une exception. `create` renvoie l'objet même s'il n'a pas été enregistré : `persisted?` le dit. Les messages passent par `I18n`, dans la langue par défaut de l'application.

Pour chaque colonne, Active Record génère aussi un prédicat (`title?`, vrai si la valeur est présente) et des méthodes de suivi (`title_was`, `title_changed?`). Les méthodes que vous écrivez s'y ajoutent ; `book.method(:title?).owner` dit qui a défini une méthode.

Un nom, un fichier. Rails charge les classes de `app/` sans `require` : le chemin du fichier donne le nom de la classe. `app/models/loan_request.rb` définit `LoanRequest`, `app/services/loans/renewal.rb` définit `Loans::Renewal`. Pour trouver une classe dans une application, convertissez son nom en chemin (`"Loans::Renewal".underscore`). En développement, le code modifié est rechargé sans redémarrer : `reload!` en console, automatiquement pour le serveur.

Seeds. `bin/rails db:seed` exécute `db/seeds.rb`. Écrit avec `create!`, il crée des doublons à chaque exécution. `find_or_create_by!` cherche d'abord (`SELECT`), et ne crée (`INSERT`) que si rien n'est trouvé :

```
Book.find_or_create_by!(title: "Les Misérables") do |book|
  book.assign_attributes(pages: 1900, copies: 2) # le bloc ne s'exécute qu'à la création
end
```

♦ Step 03 : routes, contrôleur, layout

Le cycle d'une requête

```
GET /books/3
→ config/routes.rb      resources :books : BooksController#show, avec params[:id] = "3"
→ BooksController      les before_action, puis la méthode show : @book = Book.find(params[:id])
→ la vue                pas de render : app/views/books/show.html.erb
→ le layout             la vue est insérée dans app/views/layouts/application.html.erb
→ la réponse            200, une page HTML complète
```

Le routeur associe un verbe et un chemin à un couple `contrôleur#action`. Rails crée une instance du contrôleur pour chaque requête et appelle la méthode publique qui porte le nom de l'action. `bin/rails generate controller Pages home` écrit le contrôleur, l'action, sa vue et une route.

Dans `config/routes.rb`, `root "pages#home"` envoie `GET /` vers `PagesController#home`, et `get "about", to: "pages#about"` déclare une route isolée. `bin/rails routes` liste toutes les routes, y compris celles que Rails et les gems ajoutent ; `-g` filtre sur un mot, `-C` sur un contrôleur.

Ce que Rails fait sans qu'on lui demande

- **Rendu implicite** : une action qui n'appelle ni `render` ni `redirect_to` rend la vue qui porte son nom : `PagesController#home` rend `app/views/pages/home.html.erb`.
- **Variables d'instance** : avant le rendu, Rails copie les variables d'instance du contrôleur (`@books_count = Book.count` dans l'action) dans la vue. C'est la seule interface entre les deux, sans déclaration : une variable mal orthographiée vaut `nil` dans la vue, sans erreur.

Vues, layout, partials

En ERB, `<%= ... %>` affiche le résultat en échappant le HTML, `<% ... %>` exécute sans afficher (un `if`, une boucle), et seul `<# ... %>` neutralise du Ruby : dans un commentaire HTML, `<!-- <%= ... %> -->`, le code s'exécute quand même.

Le layout `app/views/layouts/application.html.erb` enveloppe toutes les pages : la vue de l'action est insérée à la place de `<%= yield %>`. Un partial est un fragment de vue dont le nom de fichier commence par `_`, appelé sans le `_` : `<%= render "layouts/navbar" %>` rend `app/views/layouts/_navbar.html.erb`. Les helpers de Rails (`link_to`, `pluralize`...) sont disponibles dans toutes les vues.

Lire le log d'une requête

```
Started GET "/" for 127.0.0.1 at ...
Processing by PagesController#home as HTML
  Book Count (0.1ms)  SELECT COUNT(*) FROM "books"
↳ app/controllers/pages_controller.rb:3:in 'PagesController#home'
  Rendered pages/home.html.erb within layouts/application (Duration: 0.4ms | GC: 0.0ms)
  Rendered layout layouts/application.html.erb (Duration: 16.7ms | GC: 3.0ms)
Completed 200 OK in 56ms (Views: 18.0ms | ActiveRecord: 2.2ms (1 query, 0 cached) | GC: 22.4ms)
```

Dans l'ordre : la requête, le contrôleur et l'action, chaque requête SQL, les templates rendus, puis le statut et le bilan, dont le nombre de requêtes SQL. En développement, la ligne `L` donne la ligne de code qui a lancé la requête SQL au-dessus d'elle. Chaque générateur écrit aussi un test, que `bin/rails test` lance avec les autres.

♦ Step 04 : CRUD et scaffold

resources et les 7 actions

```
resources :books

GET    /books          index    books_path
GET    /books/new      new      new_book_path
POST   /books          create   books_path
GET    /books/:id      show     book_path(book)
GET    /books/:id/edit edit     edit_book_path(book)
PATCH /books/:id      update   book_path(book)      (et PUT)
DELETE /books/:id      destroy  book_path(book)
```

Sept actions, huit routes. Chaque helper `_path` a une version `_url`, avec le domaine. `link_to "Voir", book` et `redirect_to book` déduisent `book_path(book)` de la classe de l'objet.

Le code généré par le scaffold

`bin/rails generate scaffold Book title pages:integer` écrit la migration, le modèle, `resources :books`, le contrôleur, six vues (`index`, `show`, `new`, `edit`, `_form`, `_book`), un helper, des tests et des fixtures, et une API JSON (vues `.json.jbuilder`). Sur un modèle existant, `--skip` saute les fichiers qui existent déjà. Extrait du contrôleur :

```
class BooksController < ApplicationController
  before_action :set_book, only: %i[ show edit update destroy ]

  def create
    @book = Book.new(book_params)

    respond_to do |format|
      if @book.save
        format.html { redirect_to @book, notice: "Book was successfully created." }
        format.json { render :show, status: :created, location: @book }
      else
        format.html { render :new, status: :unprocessable_content }
        format.json { render json: @book.errors, status: :unprocessable_content }
      end
    end
  end
end
```

Après les autres actions (`index`, `show`, `new`, `edit`, `update`, `destroy`), les méthodes utilitaires :

```
private
def set_book
  @book = Book.find(params.expect(:id))
end

def book_params
  params.expect(book: [ :title, :pages ])
end
end
```

Un contrôleur se lit toujours dans cet ordre : les `before_action`, les actions publiques, puis sous `private` les méthodes utilitaires.

- `before_action :set_book, only: [...]` s'exécute avant les actions listées. `find` lève `RecordNotFound` si l'id n'existe pas : une 404 en production.
- `respond_to` choisit la réponse selon le format demandé : HTML pour le navigateur, JSON pour l'API générée.
- Succès : `redirect_to`. Le navigateur fait une nouvelle requête `GET`, et le message `notice` l'accompagne.
- Échec : `render :new` avec le statut 422 (`:unprocessable_content`). Le formulaire est rendu dans la même requête, avec l'objet, ses erreurs et la saisie.

Strong parameters. `params` contient tout ce que le client a envoyé, champs forgés compris. `params.expect(book: [:title, :pages])` exige une clé `book` contenant un hash, et n'en garde que les clés listées : les autres sont écartées sans erreur. Sans clé `book`, la réponse est une 400. Passer à `Book.new` des paramètres non filtrés lève `ActiveModel::ForbiddenAttributesError`.

Formulaires, partials, helpers, flash

`form_with model: @book` construit un formulaire `POST /books` pour un objet neuf, `PATCH /books/3` pour un objet enregistré (un champ caché `_method` porte le verbe). Ses champs s'appellent `book[title]`, `book[pages]` : c'est le hash que `params.expect` filtre.

`render @books` rend `app/views/books/_book.html.erb` une fois par élément, avec une variable locale `book`. Les autres variables locales se passent en hash : `render "shared/errors", record: @book` donne au partial une variable `record`, dont il affiche les erreurs (`record.errors.full_messages`).

Chaque méthode d'un module de `app/helpers/` est disponible dans toutes les vues, sans déclaration. Pour produire du HTML dans un helper, `tag.span("Disponible", class: "badge")` échappe le contenu. `<%= %>` échappe les chaînes ; un `html_safe` sur une chaîne construite avec des données saisies ouvre une faille XSS.

`redirect_to @book, notice: "Livre créé."` range le message dans le flash (`flash[:notice]`), qui survit à la redirection et disparaît après la requête suivante. Le layout l'affiche. `alert` : fonctionne de la même façon.

♦ Step 05 : relier deux modèles

belongs_to et has_many

```
bin/rails generate model Book author:references title:string copies:integer
```

Dans la migration, `author:references` devient `t.references :author, null: false, foreign_key: true` : une colonne `author_id`, un index, et une clé étrangère vers `authors`. Chaque côté de la relation se déclare dans son modèle :

```
class Book < ApplicationRecord
  belongs_to :author          # le livre porte la colonne author_id
end

class Author < ApplicationRecord
  has_many :books, dependent: :destroy
end
```

Ces deux lignes génèrent les méthodes qui lisent et écrivent la relation :

```
zola = Author.create!(name: "Émile Zola")
zola.books      # SELECT "books".* FROM "books" WHERE "books"."author_id" = 1
zola.books.count # SELECT COUNT(*) FROM "books" WHERE "books"."author_id" = 1
book.author     # SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
zola.books.create!(title: "Germinal")      # remplit author_id ; de même zola.books.build
# INSERT INTO "books" ("author_id", "title", ...) VALUES (1, 'Germinal', ...)
```

`belongs_to` rend la relation obligatoire, sans que vous écriviez de validation : `Book.new(title: "Nana").valid?` renvoie `false`, avec l'erreur « Author doit exister ». Une relation facultative s'écrit `belongs_to :author, optional: true`.

dependent: :destroy

À la suppression d'un auteur, `dependent: :destroy` charge ses livres et les détruit un par un (un `DELETE` par livre dans le log), avant de supprimer l'auteur. Sans `dependent`, la base refuse de supprimer un auteur qui a encore des livres : la clé étrangère lève `ActiveRecord::InvalidForeignKey`.

enum

Le statut d'un livre est stocké en entier, dans une colonne `t.integer :status, null: false, default: 0`. `enum` donne un nom à chaque valeur :

```
class Book < ApplicationRecord
  enum :status, { available: 0, on_loan: 1, withdrawn: 2 }
end

book.status          # => "available" : la base stocke 0
book.available?      # => true
book.on_loan!        # UPDATE "books" SET "status" = 1, ... WHERE "books"."id" = 1
Book.on_loan         # SELECT "books".* FROM "books" WHERE "books"."status" = 1
Book.statuses        # => {"available" => 0, "on_loan" => 1, "withdrawn" => 2}
Book.new(status: :lost) # ArgumentError : 'lost' is not a valid status
```

`on_loan!` écrit en base tout de suite. Écrivez le hash avec ses entiers : chaque nom garde son entier quand vous ajoutez un statut.

Routes imbriquées

Un livre se crée depuis la page de son auteur, puis a sa propre adresse :

```
resources :authors do
  resources :books, only: %i[ new create ]
end
resources :books, except: %i[ new create ]
```

GET	/authors/:author_id/books/new	books#new	new_author_book_path(author)
POST	/authors/:author_id/books	books#create	author_books_path(author)
GET	/books/:id	books#show	book_path(book)

Dans le contrôleur, l'auteur vient de l'URL, jamais du formulaire :

```
class BooksController < ApplicationController
  before_action :set_author, only: %i[ new create ]

  def create
    @book = @author.books.build(book_params) # author_id est rempli par l'association
    # ...
  end

  private
  def set_author
    @author = Author.find(params.expect(:author_id))
  end
end
```

`form_with model: [@author, @book]` envoie le formulaire d'un nouveau livre à `POST /authors/1/books` ; pour un livre existant, `form_with model: @book` l'envoie à `PATCH /books/12`.

♦ Step 06 : many-to-many avec `has_many :through`

Un modèle de jointure

Un membre emprunte plusieurs livres, et un livre est emprunté par plusieurs membres. Entre les deux, une table `loans` : chaque ligne relie un livre et un membre.

```
bin/rails generate model Member name:string email:string:uniq
bin/rails generate model Loan book:references member:references
```

```
class Loan < ApplicationRecord
  belongs_to :book
  belongs_to :member
end

class Book < ApplicationRecord # de même dans Member, avec has_many :books
  has_many :loans, dependent: :destroy
  has_many :members, through: :loans
end
```

`through` fait la jointure pour vous :

```
germinal.members
# SELECT "members".* FROM "members"
#   INNER JOIN "loans" ON "members"."id" = "loans"."member_id" WHERE "loans"."book_id" = 1
```

Deux niveaux, et `distinct`

Les lecteurs d'un auteur passent par ses livres, puis par leurs emprunts :

```
class Author < ApplicationRecord
  has_many :books, dependent: :destroy
  has_many :loans, through: :books
  has_many :members, -> { distinct }, through: :loans
end
```

`zola.members` joint `members`, `loans` et `books`, ce qui produit une ligne par emprunt : sans `distinct` (`SELECT DISTINCT, COUNT(DISTINCT ...)`), Camille, qui a emprunté deux livres de Zola, compterait deux fois.

La même règle, à deux endroits

```
class Member < ApplicationRecord
  validates :email, presence: true, uniqueness: true, format: { with: URI::MailTo::EMAIL_REGEXP }
end
```

`email:string:uniq` a aussi écrit dans la migration `add_index :members, :email, unique: true`.

- La **validation** cherche un doublon avant d'écrire (`SELECT 1 AS one FROM "members" WHERE "members"."email" = ... LIMIT 1`) et donne un message lisible : « Email est déjà utilisé(e) ».
- L'**index unique** fait refuser le doublon par la base, même quand une écriture passe à côté des validations (`save(validate: false)`, `insert_all`, du SQL écrit à la main) : `ActiveRecord::RecordNotUnique`.

Sur deux colonnes, une validation avec `scope`, et un index sur le couple :

```
validates :title, uniqueness: { scope: :author_id, message: "existe déjà pour cet auteur" }

add_index :books, [ :author_id, :title ], unique: true # dans une migration
```

L'option `message` : remplace le message par défaut : « Title existe déjà pour cet auteur ».

find_or_initialize_by

`find_or_initialize_by` cherche la ligne ; s'il ne la trouve pas, il renvoie un nouvel objet (`new_record?` vaut `true`) sans rien écrire, contrairement à `find_or_create_by!` (step 02). Vous complétez l'objet, puis `save` le valide et l'enregistre. Dans un contrôleur d'emprunt :

```
@member = Member.find_or_initialize_by(email: member_params[:email]) # SELECT ... LIMIT 1
@member.name = member_params[:name] if @member.new_record?
@loan = @book.loans.build(member: @member)

if @member.save && @loan.save
  redirect_to @book, notice: "Emprunt de #{@member.name} enregistré."
else
  # afficher @member.errors et @loan.errors
end
```

♦ Step 07 : méthode métier et validation personnalisée

Une méthode dans le modèle

```
class Book < ApplicationRecord
  def available_copies
    copies = loans.count # SELECT COUNT(*) FROM "loans" WHERE "loans"."book_id" = 1
  end
end
```

Une méthode du modèle sert partout où l'objet est disponible : contrôleur, vue (`<%= @book.available_copies %>`), console, tests.

validate et errors.add

Aucune option de `validates` ne sait compter les emprunts. La règle s'écrit dans une méthode, déclarée avec `validate` (sans `s`) :

```
class Loan < ApplicationRecord
  validate :book_has_copies, on: :create

  private
  def book_has_copies
    if book && book.available_copies <= 0
      errors.add(:book, "n'a plus d'exemplaire disponible")
    end
  end
end

loan = nana.loans.build(member: eve)
loan.valid? # => false
loan.errors.full_messages # => ["Book n'a plus d'exemplaire disponible"]
```

- `errors.add` ajoute une erreur sur un attribut : dès qu'il y a une erreur, `valid?` et `save` renvoient `false`.
- `if book && :sans livre`, `book` vaut `nil`. `belongs_to` signale déjà « Book doit exister », le test évite une `NoMethodError`.
- `on: :create` : la règle ne s'applique qu'à un nouvel emprunt. Sans elle, un emprunt déjà enregistré deviendrait invalide dès que le livre n'a plus d'exemplaire disponible.

Un bouton, pas un lien

```
resources :loans, only: :destroy # DELETE /loans/:id
```

```
<%= button_to "Rendre", loan, method: :delete,  
  form: { data: { turbo_confirm: "Rendre ce livre ?" } } %>
```

produit un petit formulaire `POST /loans/1`, avec un bouton et deux champs cachés, `_method` et `authenticity_token`.

- **Un formulaire, pas un lien** : un lien envoie un `GET`, que le navigateur peut précharger et qu'un robot peut suivre. Une action qui modifie des données passe par un formulaire.
- **`_method`** : un formulaire HTML n'envoie que `GET` ou `POST` ; Rails lit ce champ et traite la requête comme un `DELETE`.
- **`authenticity_token`** : un jeton que Rails contrôle à la réception, pour refuser les requêtes envoyées depuis un autre site (CSRF).
- **`data-turbo-confirm`** : Turbo, installé par défaut, demande confirmation avant l'envoi.

```
def destroy  
  loan = Loan.find(params.expect(:id))  
  loan.destroy!  
  redirect_to loan.book, notice: "Livre rendu.", status: :see_other  
end
```

`status: :see_other` (303) fait suivre la redirection en `GET`, quelle que soit la méthode de la requête d'origine. Le scaffold l'utilise déjà dans `update` et `destroy`.

♦ Step 08 : les callbacks

Un callback sur l'objet lui-même

Un callback est une méthode qu'Active Record appelle à un moment précis de la vie de l'objet :

```
class Member < ApplicationRecord  
  before_validation :normalize_email  
  
  private  
  def normalize_email  
    self.email = email.strip.downcase if email  
  end  
end
```

`self.email =` appelle le setter ; sans `self`, Ruby créerait une variable locale. Le callback s'exécute à chaque `valid?`, donc à chaque `save`, `create` ou `update`.

L'ordre des callbacks à la création :

```
before_validation → validations → after_validation  
  → before_save → before_create → INSERT → after_create → after_save  
  → COMMIT → after_commit
```

À la mise à jour, `before_update` et `after_update` remplacent les versions `create` ; à la suppression, `destroy` appelle `before_destroy`, le `DELETE`, puis `after_destroy`.

normalizes , aussi pour chercher

`before_validation` normalise l'email enregistré, pas celui qu'on cherche : `find_or_initialize_by(email: "Camille@Example.test")` ne trouve pas `camille@example.test`, prépare un nouveau membre, que la validation d'unicité refuse. `normalizes` s'applique à l'affectation **et** aux recherches :

```
class Member < ApplicationRecord
  normalizes :email, with: ->(email) { email.strip.downcase }
end

Member.new(email: " Eve@Example.TEST ").email      # => "eve@example.test", dès l'affectation
Member.where(email: " CAMILLE@Example.test").to_sql
# SELECT "members".* FROM "members" WHERE "members"."email" = 'camille@example.test'
```

Un callback qui modifie un autre objet

Un livre passe à « prêté » quand son dernier exemplaire part, et redevient disponible quand un exemplaire revient :

```
class Book < ApplicationRecord
  def refresh_status
    if available? && available_copies.zero?
      on_loan!
    elsif on_loan? && available_copies.positive?
      available!
    end
  end
end
```

`Loan` l'appelle après chaque création et chaque suppression :

```
class Loan < ApplicationRecord
  after_create :refresh_book_status
  after_destroy :refresh_book_status

  private
  def refresh_book_status
    book.refresh_status
  end
end
```

Dans le log d'un `loan.destroy`, le `DELETE` de l'emprunt est suivi du `SELECT` du livre, du `COUNT` de ses emprunts et, si le statut change, de son `UPDATE`. Un callback s'exécute à chaque création ou suppression, d'où qu'elle vienne : contrôleur, console, seeds, tests. Mais `delete`, `update_column` et `update_all` écrivent sans validations ni callbacks : après un `loan.delete`, le statut du livre n'est pas recalculé.

Ce que le callback cache. Rien dans `LoansController#destroy` (step 07) ne dit que le livre peut changer de statut : l'effet est déclaré dans `Loan`, loin de l'appel qui le déclenche. `normalize_email` ne touche que l'objet en cours d'enregistrement, c'est l'usage le plus sûr ; `refresh_book_status` modifie **un autre objet**, invisible pour qui lit le contrôleur. Dans un modèle que vous n'avez pas écrit, cherchez les callbacks en premier : les lignes `before_...` et `after_...`, et les `dependent:` des associations, qui en posent aussi.

♦ Step 09 : scopes et filtres

Un scope

```
class Book < ApplicationRecord
  scope :long, -> { where(pages: 500..) }
end

Book.long # SELECT "books".* FROM "books" WHERE "books"."pages" >= 500
```

Un scope donne un nom à une requête. C'est une méthode de classe qui renvoie une relation, que l'on peut compléter. Les intervalles sans borne donnent les comparaisons : `500..` pour `>= 500`, `...Time.current` pour `< Time.current`.

Enchaîner, sans requête

Les scopes s'enchaînent, avec ceux que génère un `enum`. Une relation est lazy : chaque scope ajoute une condition, et la requête ne part qu'au moment où l'on a besoin du résultat.

```
books = Book.long.available # aucune requête
books.class # => Book::ActiveRecord_Relation
books.to_sql # SELECT ... WHERE "books"."pages" >= 500 AND "books"."status" = 0
books.count # SELECT COUNT(*) FROM "books" WHERE ...
books.each { |book| ... } # SELECT "books".* FROM "books" WHERE ...
```

Un scope avec un argument

```
scope :with_status, ->(status) { where(status: status) if status.present? }
scope :by_author, ->(author_id) { where(author_id: author_id) if author_id.present? }

Book.with_status("on_loan").to_sql # SELECT "books".* FROM "books" WHERE "books"."status" = 1
Book.with_status(nil).to_sql # SELECT "books".* FROM "books"
```

Quand la condition est fausse, le bloc renvoie `nil`, et le scope renvoie alors toutes les lignes : la chaîne continue sans vérifier chaque filtre.

Filtrer une liste

Un formulaire envoyé en `GET` place ses champs dans l'URL :

```
<%= form_with url: books_path, method: :get do |form| %>
  <%= form.select :status, Book.statuses.keys,
    { include_blank: "Tous les statuts", selected: params[:status] } %>
  <%= form.collection_select :author_id, Author.order(:name), :id, :name,
    { include_blank: "Tous les auteurs", selected: params[:author_id] } %>
  <%= form.submit "Filtrer", name: nil %>
<% end %>
```

Il envoie par exemple `GET /books?status=available&author_id=1`. Le contrôleur enchaîne les scopes :

```
def index
  @books = Book.order(:title).with_status(params[:status]).by_author(params[:author_id])
end
```

Les filtres ne sont pas enregistrés en base : pas besoin de `params.expect`, chaque scope reçoit une seule valeur.

Le N+1 et `includes`

La liste affiche `book.author.name` pour chaque livre. Le log de la requête :

```
Book Load      SELECT "books".* FROM "books" ORDER BY "books"."title" ASC
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 3 LIMIT 1
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 2 LIMIT 1
CACHE Author Load SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
CACHE Author Load SELECT "authors".* FROM "authors" WHERE "authors"."id" = 2 LIMIT 1
Completed 200 OK in 32ms (Views: 19.6ms | ActiveRecord: 1.7ms (6 queries, 2 cached) | GC: 0.0ms)
```

Une requête pour la liste, puis une par ligne : c'est le **N+1**. `CACHE` signale une requête identique à une précédente, resservie sans interroger la base, le temps de la requête HTTP. Invisible avec cinq livres, coûteux avec cinq mille. `includes` charge les auteurs en même temps que les livres :

```
@books = Book.includes(:author).order(:title)
# SELECT "books".* FROM "books" ORDER BY "books"."title" ASC
# SELECT "authors".* FROM "authors" WHERE "authors"."id" IN (1, 3, 2)
```

Deux requêtes, quel que soit le nombre de livres.

♦ Step 10 : les tests

Lancer les tests

Minitest est fourni avec Rails. Les tests sont dans `test/`, un fichier par classe testée ; les générateurs en ont déjà écrit.

```
bin/rails test          # toute la suite
bin/rails test test/models/loan_test.rb  # un fichier
bin/rails test test/models/loan_test.rb:12  # le test de la ligne 12
```

Les fixtures

Les fixtures sont les données de test : un fichier YAML par table, dans `test/fixtures/`, une entrée nommée par ligne. Ici, `authors.yml` contient `zola`, et `members.yml` contient `camille` et `denis`.

```
# test/fixtures/books.yml
germinal:
  author: zola          # renvoie à l'entrée zola de authors.yml
  title: Germinal
  copies: 2
  status: available     # le nom de la valeur d'enum

# test/fixtures/loans.yml : un des deux exemplaires de Germinal est prêté
camille_germinal:
  book: germinal
  member: camille
  created_at: <%= 1.week.ago %> # l'ERB calcule une date relative
```

`books(:germinal)` renvoie le livre de l'entrée `germinal`. Les fixtures sont chargées dans une base à part, `storage/test.sqlite3`, et chaque test est annulé à la fin : les tests ne se gênent pas entre eux.

Un test de modèle, un échec

Dans `test/models/book_test.rb` :

```
require "test_helper"

class BookTest < ActiveSupport::TestCase
  test "compte les exemplaires disponibles" do
    assert_equal 1, books(:germinal).available_copies
  end
end
```

`test "..."` définit un test : il prépare une situation, agit, puis vérifie le résultat. Si `available_copies` oublie les emprunts :

```
Failure:
BookTest#test_compte_les_exemplaires_disponibles [test/models/book_test.rb:5]:
Expected: 1
Actual: 2

bin/rails test test/models/book_test.rb:4
```

Dans l'ordre : le test, le fichier et la ligne de l'assertion qui échoue, la valeur attendue et la valeur obtenue, puis la commande qui relance ce seul test. Un `assert` qui reçoit `false` affiche `Expected false to be truthy`.

Tester les règles

Dans `test/models/loan_test.rb`, après `require "test_helper"` :

```
class LoanTest < ActiveSupport::TestCase
  test "un livre sans exemplaire disponible ne peut pas être emprunté" do
    book = books(:germinal)
    book.update!(copies: 1)
    loan = Loan.new(book: book, member: members(:denis))

    assert_not loan.valid?
    assert_includes loan.errors[:book], "n'a plus d'exemplaire disponible"
  end

  test "rendre un exemplaire repasse le livre à disponible" do
    book = books(:germinal)
    book.loans.create!(member: members(:denis)) # le dernier exemplaire part : on_loan

    loans(:camille_germinal).destroy

    assert book.reload.available?
  end
end
```

`loans(:camille_germinal)` charge son livre dans un autre objet Ruby : le callback `after_destroy` modifie cet objet-là, pas la variable `book`. `reload` relit la ligne en base. Sans lui, le test échoue.

Les assertions courantes : `assert` et `assert_not` (la valeur est `truthy`, ou `falsy`), `assert_equal` attendu, obtenu, `assert_nil`, `assert_includes` collection, élément, `assert_difference("Loan.count") do ... end` (le compte augmente de 1 pendant le bloc ; -1 en second argument pour une baisse), `assert_no_difference`, et pour les requêtes `assert_redirected_to` et `assert_select`.

Un test d'intégration

Un test d'intégration envoie de vraies requêtes à l'application, sans navigateur : routeur, contrôleur, modèle et vue.

```
# test/controllers/loans_controller_test.rb
require "test_helper"

class LoansControllerTest < ActionDispatch::IntegrationTest
  test "emprunter un livre" do
    book = books(:germinal)

    assert_difference("Loan.count") do
      post book_loans_url(book),
        params: { member: { name: "Ève Leroy", email: "eve@example.test" } }
    end

    assert_redirected_to book_url(book)
    follow_redirect!
    assert_select "td", "eve@example.test"
  end
end
```

`params:` reprend la forme des champs du formulaire (`member[name]`, `member[email]`). `follow_redirect!` suit la redirection; `assert_select "td", "eve@example.test"` cherche une cellule avec ce texte dans la page. `delete loan_url(loans(:camlille_germinal))` envoie la requête du bouton « Rendre ». Après la requête, `flash[:alert]` donne le message d'erreur éventuel.

♦ Step 11 : association polymorphe et concern

Une table pour deux modèles

Un avis peut porter sur un livre ou sur un auteur. Plutôt qu'une table d'avis par modèle, une seule table, dont chaque ligne dit à quel type d'objet elle appartient :

```
bin/rails generate model Review body:text "reviewable:references{polymorphic}"
```

La migration contient `t.references :reviewable, polymorphic: true, null: false` : deux colonnes, `reviewable_type` (le nom de la classe, "Book" ou "Author") et `reviewable_id` (l'id dans la table correspondante), et un index sur le couple.

```
class Review < ApplicationRecord
  belongs_to :reviewable, polymorphic: true
end

class Book < ApplicationRecord
  # de même dans Author
  has_many :reviews, as: :reviewable, dependent: :destroy
end
```

```
review = germinal.reviews.create!(body: "Le meilleur roman de la série")
review.reviewable_type # => "Book"
review.reviewable      # => le livre : Rails lit reviewable_type pour choisir la classe
germinal.reviews
# SELECT "reviews".* FROM "reviews"
# WHERE "reviews"."reviewable_id" = 1 AND "reviews"."reviewable_type" = 'Book'
```

`as: :reviewable` indique à `has_many` de chercher par les deux colonnes. `reviewable_id` pointe tantôt vers `books`, tantôt vers `authors` : aucune clé étrangère n'est possible, et l'intégrité repose sur le code. `belongs_to` vérifie le parent à l'enregistrement (« Reviewable doit exister »), `dependent: :destroy` supprime les avis avec leur livre.

Un contrôleur pour deux parents

Dans `config/routes.rb`, `resources :reviews, only: :create`, placé dans le bloc `resources :authors` et dans le bloc `resources :books`, donne deux URL, `POST /authors/:author_id/reviews` et `POST /books/:book_id/reviews`, pour un seul contrôleur. Le paramètre présent dit quel parent charger :

```
class ReviewsController < ApplicationController
  before_action :set_reviewable

  def create
    review = @reviewable.reviews.build(review_params)
    if review.save
      redirect_to @reviewable, notice: "Avis ajouté."
    else
      redirect_to @reviewable, alert: "Un avis ne peut pas être vide."
    end
  end

  private
  def set_reviewable
    if params[:book_id]
      @reviewable = Book.find(params[:book_id])
    else
      @reviewable = Author.find(params[:author_id])
    end
  end

  def review_params
    params.expect(review: [ :body ])
  end
end
```

`create` n'a pas besoin de savoir si `@reviewable` est un livre ou un auteur : les deux ont `reviews`, et `redirect_to` trouve l'URL de chacun. Même chose dans un partial partagé, rendu avec `render "reviews/reviews", reviewable: @book : form_with model: [reviewable, Review.new]` envoie le formulaire à `/books/1/reviews` ou à `/authors/1/reviews`.

Un concern pour le code partagé

`Book` et `Author` contiennent la même ligne `has_many :reviews, as: :reviewable, dependent: :destroy`. Un concern la range dans un module, écrit une seule fois :

```
# app/models/concerns/reviewable.rb
module Reviewable
  extend ActiveSupport::Concern

  included do
    has_many :reviews, as: :reviewable, dependent: :destroy
  end

  def reviewed?
    reviews.any?
  end

  class Book < ApplicationRecord # de même dans Author
    include Reviewable
  end
end
```

`has_many` est une méthode de classe : elle doit s'exécuter dans la classe qui inclut le module. C'est le rôle de `included do`, dont le bloc s'exécute dans `Book`, puis dans `Author`, au moment du `include`. Les méthodes écrites hors du bloc deviennent des méthodes d'instance, comme dans tout module. `Book.ancestors` liste `Reviewable` juste après `Book`.

Les concerns de modèles sont dans `app/models/concerns/`, ceux de contrôleurs dans `app/controllers/concerns/`.

♦ Step 12 : Active Storage

Installer

```
bin/rails active_storage:install
bin/rails db:migrate
```

La migration crée trois tables, communes à tous les modèles, dont `active_storage_blobs` (un fichier : nom, type, taille, clé de stockage) et `active_storage_attachments` (le lien entre un enregistrement et un fichier, par `record_type` et `record_id` : une association polymorphe, comme au step 11).

Joindre un fichier

```
class Book < ApplicationRecord
  has_one_attached :cover          # has_many_attached pour plusieurs fichiers
end
```

Aucune colonne dans `books`. Le contrôleur accepte le champ (`params.expect(book: [ :title, :copies, :cover ])`), le formulaire l'envoie, la page l'affiche :

```
<%= form.file_field :cover, accept: "image/png,image/jpeg" %>

<% if @book.cover.attached? %>
  <%= image_tag @book.cover %>
  <%= link_to "Télécharger", rails_blob_path(@book.cover, disposition: "attachment") %>
<% end %>
```

`form_with` passe seul le formulaire en `multipart/form-data` dès qu'il contient un `file_field`. `rails_blob_path` donne une URL (`/rails/active_storage/blobs/redirect/.../germinal.png`) qui redirige vers le fichier ; `disposition: "attachment"` le fait télécharger plutôt qu'afficher.

En console ou dans les seeds :

```
file = File.open(Rails.root.join("db/seeds/germinal.png"))
book.cover.attach(io: file, filename: "germinal.png")
book.cover.content_type      # => "image/png" ; aussi filename, byte_size, attached?
```

Où est le fichier ?

`config/storage.yml` déclare les services de stockage, et chaque environnement choisit le sien (`config.active_storage.service = :local` dans `config/environments/development.rb`). En développement, le service `local` écrit les fichiers dans `storage/`, sous un nom aléatoire (`storage/wv/sk/wvsk37g27pq98pwca9a8f5mby69t`) ; la base ne garde que les informations sur le fichier. En production, un autre service (Amazon S3, Google Cloud Storage, un disque partagé...) se déclare dans le même fichier, sans toucher au code de l'application.

Valider le type

`accept:` ne filtre que la fenêtre de sélection du navigateur, il ne vérifie rien côté serveur. Une validation, comme au step 07 :

```
class Book < ApplicationRecord
  validate :cover_is_an_image

  private
  def cover_is_an_image
    if cover.attached? && ![image/png image/jpeg].include?(cover.content_type)
      errors.add(:cover, "doit être une image PNG ou JPEG")
    end
  end
end
```

Un PDF envoyé à la place d'une image est refusé : « Cover doit être une image PNG ou JPEG ».

♦ Step 13 : Turbo Drive et Turbo Frames

Turbo Drive

Turbo est installé par défaut (`app/javascript/application.js` importe `@hotwired/turbo-rails`). Turbo Drive intercepte les clics sur les liens et les envois de formulaires, fait la requête lui-même et remplace le `<body>` par celui de la réponse : la page n'est jamais rechargée en entier.

Après l'envoi d'un formulaire, Turbo attend une redirection (le succès) ou un statut d'erreur (l'échec) : c'est la raison du `status: :unprocessable_content` (422) des `render` de `create` et `update` (step 04). Sans lui, Turbo n'affiche rien et signale dans la console du navigateur : `Form responses must redirect to another location`.

Un frame : une zone qui se met à jour seule

```
<%= turbo_frame_tag "book_details" do %>
  <h1><%= @book.title %></h1>
  <p><%= pluralize(@book.copies, "exemplaire", plural: "exemplaires") %></p>

  <%= link_to "Modifier", edit_book_path(@book) %>
<% end %>
```

```
<%= turbo_frame_tag "book_details" do %>
  <%= render "form", book: @book %>

  <%= link_to "Annuler", @book %>
<% end %>
```

`turbo_frame_tag "book_details"` produit une balise `<turbo-frame id="book_details">`. Un lien ou un formulaire placé **dans** un frame ne change pas de page : Turbo fait la requête, cherche dans la réponse le frame qui porte le même `id`, et remplace le contenu du frame affiché par le sien. Le reste de la réponse est ignoré.

Ici, « Modifier » affiche le formulaire à la place des informations ; l'enregistrement redirige vers `show`, dont Turbo ne garde que le frame ; une erreur réaffiche le formulaire sur place. Le contrôleur ne change pas.

Sortir d'un frame

Si la réponse ne contient pas le frame attendu, Turbo affiche « Content missing ». Pour qu'un lien placé dans un frame change toute la page :

```
<%= link_to "Voir l'auteur", @book.author, data: { turbo_frame: "_top" } %>
```

Un frame ne met à jour que lui-même : ce qui est affiché ailleurs dans la page (un titre, un compteur, un message du layout) garde son ancienne valeur.

♦ Step 14 : Turbo Streams

Plusieurs zones, une réponse

Un Turbo Stream est une réponse faite d'instructions. Chacune vise un élément de la page par son `id` :

```
<td id="available_copies"><%= @book.available_copies %></td>
<tbody id="loans"><%= render @book.loans %></tbody>
```

Le contrôleur répond en stream quand la requête vient de Turbo, et garde une réponse HTML pour les autres (tests, requêtes sans JavaScript) :

```
def create
  # ... @loan est construit et enregistré
  respond_to do |format|
    format.turbo_stream # rend create.turbo_stream.erb
    format.html { redirect_to @book, notice: "Emprunt enregistré." }
  end
end
```

```
<%# app/views/loans/create.turbo_stream.erb %>
<%= turbo_stream.append "loans", @loan %>
<%= turbo_stream.update "available_copies", @book.available_copies %>
```

La réponse est du HTML, une balise par instruction (`<turbo-stream action="append" target="loans">`), à lire dans l'onglet Réseau du navigateur. `turbo_stream.append "loans", @loan` rend le partial `loans/_loan`, comme `render @loan`.

Action	Effet
append / prepend	ajoute à la fin / au début de la cible
update	remplace le contenu de la cible, garde la balise
replace	remplace la cible elle-même, balise comprise
remove	supprime la cible : <code>turbo_stream.remove @loan</code> vise l'élément <code>loan_12</code>

Pour rendre un partial dans une instruction : `turbo_stream.replace "loan_form", partial: "loans/form", locals: { book: @book }`. Une instruction dont la cible n'existe pas dans la page est ignorée.

Prévenir les autres navigateurs

Une page ouverte ailleurs n'a rien demandé : le serveur doit lui pousser le changement, par une connexion qui reste ouverte (un WebSocket, géré par Action Cable).

```
<%# app/views/books/show.html.erb : la page s'abonne aux messages de ce livre %>
<%= turbo_stream_from @book %>
```

```
class Loan < ApplicationRecord
  belongs_to :book
  broadcasts_refreshes_to :book # après chaque création, modification ou suppression
end
```

Les pages abonnées reçoivent une instruction `refresh` et rechargent leur contenu. `broadcasts_refreshes_to` pose des callbacks `after_commit` : comme ceux du step 08, ils s'exécutent d'où que vienne l'écriture.

Par défaut, ce rafraîchissement remplace toute la page, et la fenêtre remonte en haut. Avec le **morphing**, Turbo compare la nouvelle page avec celle affichée et ne modifie que ce qui a changé :

```
<%# en haut de books/show.html.erb ; le layout place ce bloc dans le <head> avec yield :head %>
<% content_for :head do %>
  <%= turbo_refreshes_with method: :morph, scroll: :preserve %>
<% end %>
```

C'est l'approche inverse des streams : le serveur renvoie la page entière, Turbo calcule la différence. Une saisie en cours dans la page rafraîchie est perdue.

♦ Step 15 : Stimulus

Le serveur rend toujours le HTML. Stimulus y accroche un peu de JavaScript, pour ce qui se passe dans le navigateur sans demander de page : envoyer un formulaire dès qu'un filtre change, compter les caractères pendant la saisie. Turbo et Stimulus forment Hotwire, installé par défaut.

Un contrôleur, un fichier

`bin/rails generate stimulus auto_submit` écrit `app/javascript/controllers/auto_submit_controller.js`, avec une méthode `connect()` vide, que l'on remplace :

```
import { Controller } from "@hotwired/stimulus"

// Connects to data-controller="auto-submit"
export default class extends Controller {
  submit() {
    this.element.requestSubmit()
  }
}
```

Le nom du fichier donne le nom du contrôleur : `auto_submit_controller.js` devient `auto-submit`. Rien à déclarer : `config/importmap.rb` rend disponibles tous les fichiers du dossier (`pin_all_from "app/javascript/controllers"`), et `app/javascript/controllers/index.js` les enregistre tous (`eagerLoadControllersFrom`). Le `hello_controller.js` créé avec l'application est un exemple.

data-controller et data-action

Le formulaire de filtres du step 09, envoyé dès qu'un `select` change :

```
<%= form_with url: books_path, method: :get,
  data: { controller: "auto-submit", action: "change->auto-submit#submit" } do |form| %>
  <%# les select du step 09 %>
<% end %>
```

- `data-controller="auto-submit"` attache une instance du contrôleur au formulaire, qui devient `this.element`.
- `data-action` se lit `événement->contrôleur#méthode` : quand un champ du formulaire change, Stimulus appelle `submit()`.
- `requestSubmit()` envoie le formulaire comme un clic sur son bouton : Turbo le prend en charge, sans recharger la page. `submit()` rechargerait la page entière.

Targets, `connect()` et values

Un compteur sous le champ d'un avis (`bin/rails generate stimulus character_counter`):

```
import { Controller } from "@hotwired/stimulus"

export default class extends Controller {
  static targets = [ "input", "counter" ]

  connect() {
    this.update()
  }

  update() {
    const length = this.inputTarget.value.length
    this.counterTarget.textContent = `${length} / ${this.inputTarget.maxLength}`
  }
}
```

```
<div data-controller="character-counter">
  <%= form.textarea :body, maxlength: 500,
    data: { character_counter_target: "input", action: "input->character-counter#update" } %>
  <small data-character-counter-target="counter"></small>
</div>
```

- Une target est un élément dont le contrôleur a besoin : déclarée dans `static targets`, marquée dans le HTML par `data-character-counter-target="counter"` (dans `data:`, Rails change les `_` en `-`), lue avec `this.counterTarget`. Elle doit se trouver dans l'élément qui porte `data-controller`.
- L'événement `input` part à chaque frappe : `update()` réécrit le compteur, « 7 / 500 ».
- `connect()` s'exécute quand le contrôleur s'attache à l'élément, y compris après une navigation Turbo : le compteur affiche « 0 / 500 » avant toute saisie.
- Une value passe une donnée du HTML au contrôleur : `static values = { max: Number }` lit l'attribut `data-character-counter-max-value="500"`, converti en nombre, dans `this.maxValue`.

Si rien ne se passe, ouvrez la console du navigateur. Une target absente ou une méthode inconnue y affiche une erreur (`Missing target element "counter" for "character-counter" controller`). Une faute dans `data-controller` n'en affiche aucune : le contrôleur ne s'attache pas, c'est tout.

♦ Production et sécurité

Environnements. `RAILS_ENV` (`development`, `test` ou `production`) choisit le fichier `config/environments/<environnement>.rb` et la section de `config/database.yml`. En production, le code n'est pas rechargé à chaque modification.

Secrets. `bin/rails credentials:edit` ouvre `config/credentials.yml.enc`, chiffré et commité. La clé qui le déchiffre, `config/master.key`, n'est jamais commitée ; en production, elle passe par la variable `RAILS_MASTER_KEY`. Le code lit un secret avec `Rails.application.credentials.dig(:payment, :api_key)`. Les paramètres sensibles apparaissent en `[FILTERED]` dans les journaux (liste dans `config/initializers/filter_parameter_logging.rb`).

Ce que Rails protège par défaut. Les requêtes forgées depuis un autre site (CSRF), par l' `authenticity_token` des formulaires (step 07). L'injection de HTML ou de JavaScript (XSS), par l'échappement de `<%= %>`, que `raw` et `html_safe` désactivent (step 04). L'affectation de champs non prévus, par `params.expect` (step 04).

Ce qu'il ne protège pas : l'injection SQL. Une donnée reçue ne s'interpole jamais dans une chaîne SQL :

```
Book.where("title LIKE '#{params[:q]}%'") # injectable
Book.where("title LIKE ?", "%#{params[:q]}%") # Rails échappe la valeur
Book.where(title: params[:q]) # la forme hash est sûre
```

Les outils. Le `Gemfile` d'une application Rails 8.1 contient déjà `brakeman` et `bundler-audit`. `bin/brakeman` analyse le code à la recherche de failles connues (injection SQL, XSS...); `bin/bundler-audit` compare les versions du `Gemfile.lock` à une base de failles publiées.

Déploiement. Une application Rails 8 est générée avec un `Dockerfile` et une configuration Kamal (`config/deploy.yml`): `kamal deploy` construit l'image et la déploie sur vos serveurs. C'est un exemple parmi d'autres ; chaque hébergement a sa procédure.

♦ À reconnaître en lisant du code

Des notions que vous rencontrerez dans d'autres applications Rails :

Vous voyez	Ce que c'est
un callback que le modèle ne déclare pas	une macro ou une gem peut en poser : <code>has_one_attached</code> ajoute un <code>after_save</code> et un <code>after_commit</code> . <code>Book._save_callbacks</code> et <code>Book._commit_callbacks</code> les listent
<code>app/mailers/</code> , <code>deliver_now</code> , <code>deliver_later</code>	Action Mailer : une classe par famille de mails, une méthode et des vues par mail, comme un contrôleur
<code>app/jobs/</code> , <code>perform_later</code>	Active Job : un traitement exécuté plus tard, hors de la requête ; en production, Rails 8 les confie à Solid Queue
<code>belongs_to :book</code> , <code>counter_cache: true</code>	Rails tient à jour le nombre d'emprunts dans une colonne <code>books.loans_count</code>
<code>Book.transaction do ... end</code> , <code>book.with_lock do ... end</code>	une transaction : toutes les écritures du bloc, ou aucune. <code>with_lock</code> verrouille en plus la ligne pendant le bloc

♦ Pour aller plus loin : la documentation officielle

Step 01 - L'application et la découverte d'Active Record

- [Getting Started with Rails](https://guides.rubyonrails.org/getting_started.html)
guides.rubyonrails.org/getting_started.html
- [Active Record Migrations](https://guides.rubyonrails.org/active_record_migrations.html)
guides.rubyonrails.org/active_record_migrations.html
- [Active Record Basics](https://guides.rubyonrails.org/active_record_basics.html)
guides.rubyonrails.org/active_record_basics.html
- [Active Record Query Interface](https://guides.rubyonrails.org/active_record_querying.html)
guides.rubyonrails.org/active_record_querying.html
- [Active Record Basics - Active Record Pattern](https://guides.rubyonrails.org/active_record_basics.html#the-active-record-pattern)
guides.rubyonrails.org/active_record_basics.html#the-active-record-pattern
- [Dirty tracking](https://api.rubyonrails.org/classes/ActiveModel/Dirty.html)
api.rubyonrails.org/classes/ActiveModel/Dirty.html

Step 02 - Enrichir Workshop, comprendre où sont branchées les choses

- [Active Record Migrations](https://guides.rubyonrails.org/active_record_migrations.html#creating-a-standalone-migration)
guides.rubyonrails.org/active_record_migrations.html#creating-a-standalone-migration
- [Active Record Validations](https://guides.rubyonrails.org/active_record_validations.html)
guides.rubyonrails.org/active_record_validations.html
- [I18n - Active Record Models](https://guides.rubyonrails.org/i18n.html#active-record-models)
guides.rubyonrails.org/i18n.html#active-record-models
- [ActiveRecord::AttributeMethods::Query](https://api.rubyonrails.org/classes/ActiveRecord/AttributeMethods/Query.html)
api.rubyonrails.org/classes/ActiveRecord/AttributeMethods/Query.html
- [Autoloading and Reloading Constants](https://guides.rubyonrails.org/autoloading_and_reloading_constants.html)
guides.rubyonrails.org/autoloading_and_reloading_constants.html
- [Seeds](https://guides.rubyonrails.org/active_record_migrations.html#migrations-and-seed-data)
guides.rubyonrails.org/active_record_migrations.html#migrations-and-seed-data
- [find_or_create_by](https://api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_create_by)
api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_create_by

Step 03 - Routes, contrôleur, layout

- [Rails Routing from the Outside In](https://guides.rubyonrails.org/routing.html)
guides.rubyonrails.org/routing.html
- [Layouts and Rendering](https://guides.rubyonrails.org/layouts_and_rendering.html#rendering-by-default-convention-over-configuration-in-action)
guides.rubyonrails.org/layouts_and_rendering.html#rendering-by-default-convention-over-configuration-in-action
- [Using Partial](https://guides.rubyonrails.org/layouts_and_rendering.html#using-partials)s
guides.rubyonrails.org/layouts_and_rendering.html#using-partials
- [Bootstrap Navbar](https://getbootstrap.com/docs/5.3/components/navbar/)
getbootstrap.com/docs/5.3/components/navbar/

Step 04 - CRUD des ateliers avec le scaffold

- [Generating Code](https://guides.rubyonrails.org/command_line.html#generating-code)
guides.rubyonrails.org/command_line.html#generating-code
- [Resource Routing](https://guides.rubyonrails.org/routing.html#resource-routing-the-rails-default)
guides.rubyonrails.org/routing.html#resource-routing-the-rails-default
- [Strong Parameters](https://guides.rubyonrails.org/action_controller_overview.html#strong-parameters)
guides.rubyonrails.org/action_controller_overview.html#strong-parameters
- [Action View Helpers](https://guides.rubyonrails.org/action_view_helpers.html)
guides.rubyonrails.org/action_view_helpers.html
- [The Flash](https://guides.rubyonrails.org/action_controller_overview.html#the-flash)
guides.rubyonrails.org/action_controller_overview.html#the-flash
- [Displaying Validation Errors in Views](https://guides.rubyonrails.org/active_record_validations.html#displaying-validation-errors-in-views)
guides.rubyonrails.org/active_record_validations.html#displaying-validation-errors-in-views

Step 05 - Les sessions d'un atelier

- [Active Record Migrations - Column Modifiers](https://guides.rubyonrails.org/active_record_migrations.html#column-modifiers)
guides.rubyonrails.org/active_record_migrations.html#column-modifiers
- [Active Record Associations](https://guides.rubyonrails.org/association_basics.html)
guides.rubyonrails.org/association_basics.html
- [ActiveRecord::Enum](https://api.rubyonrails.org/classes/ActiveRecord/Enum.html)
api.rubyonrails.org/classes/ActiveRecord/Enum.html
- [Nested Resources](https://guides.rubyonrails.org/routing.html#nested-resources)
guides.rubyonrails.org/routing.html#nested-resources
- [Action Controller Overview](https://guides.rubyonrails.org/action_controller_overview.html)
guides.rubyonrails.org/action_controller_overview.html

Step 06 - Participants et inscriptions

- [The has_many :through Association](https://guides.rubyonrails.org/association_basics.html#has-many-through)
guides.rubyonrails.org/association_basics.html#has-many-through
- [Uniqueness](https://guides.rubyonrails.org/active_record_validations.html#uniqueness)
guides.rubyonrails.org/active_record_validations.html#uniqueness
- [find_or_initialize_by \(API\)](https://api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_initialize_by)
api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_initialize_by

Step 07 - La jauge

- [Custom Methods](https://guides.rubyonrails.org/active_record_validations.html#custom-methods)
guides.rubyonrails.org/active_record_validations.html#custom-methods
- [errors.add](https://api.rubyonrails.org/classes/ActiveModel/Errors.html#method-i-add)
api.rubyonrails.org/classes/ActiveModel/Errors.html#method-i-add
- [button_to](https://api.rubyonrails.org/classes/ActionView/Helpers/UrlHelper.html#method-i-button_to)
api.rubyonrails.org/classes/ActionView/Helpers/UrlHelper.html#method-i-button_to
- [Turbo - Requiring Confirmation](https://turbo.hotwired.dev/handbook/drive#requiring-confirmation-for-a-visit)
turbo.hotwired.dev/handbook/drive#requiring-confirmation-for-a-visit

Step 08 - Les callbacks

- [Active Record Callbacks](https://guides.rubyonrails.org/active_record_callbacks.html)
guides.rubyonrails.org/active_record_callbacks.html
- [normalizes \(API\)](https://api.rubyonrails.org/classes/ActiveModel/Attributes/Normalization/ClassMethods.html#method-i-normalizes)
api.rubyonrails.org/classes/ActiveModel/Attributes/Normalization/ClassMethods.html#method-i-normalizes
- [Available Callbacks](https://guides.rubyonrails.org/active_record_callbacks.html#available-callbacks)
guides.rubyonrails.org/active_record_callbacks.html#available-callbacks

Step 09 - Scopes et filtres

- [Scopes](https://guides.rubyonrails.org/active_record_querying.html#scopes)
guides.rubyonrails.org/active_record_querying.html#scopes
- [Form Helpers - A Generic Search Form](https://guides.rubyonrails.org/form_helpers.html#a-generic-search-form)
guides.rubyonrails.org/form_helpers.html#a-generic-search-form
- [N+1 Queries Problem](https://guides.rubyonrails.org/active_record_querying.html#n-1-queries-problem)
guides.rubyonrails.org/active_record_querying.html#n-1-queries-problem
- [Eager Loading Associations](https://guides.rubyonrails.org/active_record_querying.html#eager-loading-associations)
guides.rubyonrails.org/active_record_querying.html#eager-loading-associations

Step 10 - Les tests

- [Testing Rails Applications](https://guides.rubyonrails.org/testing.html)
guides.rubyonrails.org/testing.html
- [Fixtures](https://guides.rubyonrails.org/testing.html#fixtures)
guides.rubyonrails.org/testing.html#fixtures
- [Testing Models](https://guides.rubyonrails.org/testing.html#testing-models)
guides.rubyonrails.org/testing.html#testing-models
- [Minitest assertions](https://docs.seattlerb.org/minitest/Minitest/Assertions.html)
docs.seattlerb.org/minitest/Minitest/Assertions.html
- [Integration Testing](https://guides.rubyonrails.org/testing.html#integration-testing)
guides.rubyonrails.org/testing.html#integration-testing

Step 11 - Des notes sur les ateliers et les sessions

- [Polymorphic Associations](https://guides.rubyonrails.org/association_basics.html#polymorphic-associations)
guides.rubyonrails.org/association_basics.html#polymorphic-associations
- [ActiveSupport::Concern](https://api.rubyonrails.org/classes/ActiveSupport/Concern.html)
api.rubyonrails.org/classes/ActiveSupport/Concern.html

Step 12 - Un support PDF par atelier

- [Active Storage Overview](https://guides.rubyonrails.org/active_storage_overview.html)
guides.rubyonrails.org/active_storage_overview.html
- [has_one_attached](https://guides.rubyonrails.org/active_storage_overview.html#attaching-files-to-records-has-one-attached)
guides.rubyonrails.org/active_storage_overview.html#attaching-files-to-records-has-one-attached
- [Serving Files](https://guides.rubyonrails.org/active_storage_overview.html#serving-files)
guides.rubyonrails.org/active_storage_overview.html#serving-files

Step 13 - Turbo Frames : modifier sur place

- [Turbo Drive - Form Submissions](https://turbo.hotwired.dev/handbook/drive#form-submissions)
turbo.hotwired.dev/handbook/drive#form-submissions
- [Turbo Frames](https://turbo.hotwired.dev/handbook/frames)
turbo.hotwired.dev/handbook/frames
- [Targeting Navigation Into or Out of a Frame](https://turbo.hotwired.dev/handbook/frames#targeting-navigation-into-or-out-of-a-frame)
turbo.hotwired.dev/handbook/frames#targeting-navigation-into-or-out-of-a-frame

Step 14 - Turbo Streams : s'inscrire sans recharger la page

- [Turbo Streams](https://turbo.hotwired.dev/handbook/streams)
turbo.hotwired.dev/handbook/streams
- [Streaming From HTTP Responses](https://turbo.hotwired.dev/handbook/streams#streaming-from-http-responses)
turbo.hotwired.dev/handbook/streams#streaming-from-http-responses
- [Broadcasting Page Refreshes](https://turbo.hotwired.dev/handbook/page_refreshes#broadcasting-page-refreshes)
turbo.hotwired.dev/handbook/page_refreshes#broadcasting-page-refreshes
- [Page Refreshes](https://turbo.hotwired.dev/handbook/page_refreshes)
turbo.hotwired.dev/handbook/page_refreshes

Step 15 - Stimulus : un peu de JavaScript, accroché au HTML

- [Stimulus Handbook](https://stimulus.hotwired.dev/handbook/introduction)
stimulus.hotwired.dev/handbook/introduction
- [Actions](https://stimulus.hotwired.dev/reference/actions)
stimulus.hotwired.dev/reference/actions
- [Targets](https://stimulus.hotwired.dev/reference/targets)
stimulus.hotwired.dev/reference/targets
- [Lifecycle Callbacks](https://stimulus.hotwired.dev/reference/lifecycle-callbacks)
stimulus.hotwired.dev/reference/lifecycle-callbacks

- Values

stimulus.hotwired.dev/reference/values